

Contents

1	프로그래밍 언어	3
1.1	2025 경북대 3-1번	4
1.2	2025 경북대 3-2번	6
1.3	2025 경북대 3-3번	9
1.4	2025 경북대 3-4번	11
1.5	2025 경북대 3-5번	13
1.6	2024 경북대 3-1번	15
1.7	2024 경북대 3-2번	17
1.8	2024 경북대 3-3번	19
1.9	2024 경북대 3-4번	21
1.10	2024 경북대 3-5번	23
1.11	2024 경북대 3-6번	25
1.12	2023 경북대 3-1번	27
1.13	2023 경북대 3-2번	29
1.14	2023 경북대 3-3번	31
1.15	2023 경북대 3-4번	33
1.16	2022 경북대 3-1번	35
1.17	2022 경북대 3-2번	36
1.18	2021 경북대 3-1번	40
1.19	2021 경북대 3-2번	42
1.20	2021 경북대 3-3번	44
1.21	2020 경북대 3-1번	47

1.22 2020 경북대 3-2번	49
1.23 2020 경북대 3-3번	51

Chapter 1

프로그래밍 언어

1.1 2025 경북대 3-1번

1. 다음은 n 값의 자릿수가 0인지, 1자리인지, 2자리 이상인지 판정해서 출력하는 프로그램의 일부분이다.

```
if ( n == 0 )
    printf("0입니다.\n");
else if ( ( n >= -9 [2]_____ n <= 9 ) )
    printf("1자리입니다.\n");
else
    printf("2자리 이상입니다.\n");
```

1. Answer:

```
if ( n == 0 )
    printf("0입니다.\n");
else if ( ( n >= -9 && n <= 9 ) )
    printf("1자리입니다.\n");
else
    printf("2자리 이상입니다.\n");
```

2. Application example.

```
#include <stdio.h>

int main() {
    // Write C code here
    int n;

    printf("정수를 입력하세요..");
    scanf("%d", &n); // 키보드 입력을 n에 저장
    // 조건문 작성 (문제에서 주어진 로직)
    if ( n == 0 )
        printf("0입니다.\n");
    else if ( n >= -9 && n <= 9 )
        printf("1자리입니다.\n");
    else
        printf("2자리 이상입니다.\n");

    return 0;
}
```

1.2 2025 경북대 3-2번

2. 다음은 3행 4열의 2차원 행렬 'b'의 모든 구성 요소를 'a'로 복사하는 프로그램의 일부이다.

```
int main() {  
    int i, j; int a[3][4];  
    int b[3][4] = {{1,2,3,4}, {6,7,8,9}, {11,12,13,14}};  
    for (i=0; i<3; i++)  
        for (j=0; j<4; [5]_____ )  
            a[i][j] = [7]_____;  
}
```

1. Answer:

```
int main() {
    int i, j;
    int a[3][4];

    int b[3][4] = {{1,2,3,4}, {6,7,8,9}, {11,12,13,14}};

    for (i=0; i<3; i++)
        for (j=0; j<4; j++)
            a[i][j] = b[i][j];
}
```

2. Application Example:

```
#include <stdio.h>

int main(void) {
    int i, j;
    int a[3][4];
    int b[3][4] = {
        {1, 2, 3, 4},
        {6, 7, 8, 9},
        {11, 12, 13, 14}
    };

    // 복사
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 4; j++) {
            a[i][j] = b[i][j];
        }
    }
}
```

```
// 출력
printf("배열 b의 내용 (3×4):\n");
for (i = 0; i < 3; i++) {
    for (j = 0; j < 4; j++) {
        printf("%d ", b[i][j]);
    }
    printf("\n");
}

printf("\n배열 a의 내용 (복사된 결과):\n");
for (i = 0; i < 3; i++) {
    for (j = 0; j < 4; j++) {
        printf("%d ", a[i][j]);
    }
    printf("\n");
}

return 0;
}
```

1.3 2025 경북대 3-3번

3. 주어진 문자열에서 숫자를 삭제하는 함수 `del_digits()`의 빈 칸을 채워 완성하여라.

```
void del_digits( char *str ) {
    char *ptr = str;
    while (*str != [8]_____ ) {
        if (*str < '0' || *str > [9]_____ )
            *ptr++ = [10]_____;
        str++;
    }
    *ptr = [11]_____;
}

int main(void) {
    char str[256] = "A1B2C933D";
    printf("str = \"%s\" --> ", str);
    del_digits(str);
    printf("\"%s\"\\n", str);
    // 출력 --> "ABCD"
}
```

Solution:

```
void del_digits(char *str) {
    char *ptr = str;
    while (*str != '\0') {
        // (8) 문자열이 끝날 때까지 반복해야 하므로, 문자열 종료 문자인 '\0'을 조건으로 사용

        if (*str < '0' || *str > '9')
            // (9) 현재 문자가 '0'보다 작거나 '9'보다 크면 숫자가 아님

            *ptr++ = *str;
            // (10) 숫자가 아닌 문자를 앞으로 복사하고, ptr을 다음 위치로 이동

        str++;          // 다음 문자 검사
    }
    *ptr = '\0';
    // (11) 복사가 끝난 후 문자열의 끝을 '\0'으로 닫아줌
}
```

1.4 2025 경북대 3-4번

4. 다음은 'x'를 'n'번 곱한 값을 반환하는 함수다.

```
double power1(double x, int n) {  
    // returns x*x*...*x (n times)  
  
    int i;  
    double tmp;  
    for (i=0, tmp= [12]_____ ; i<n; [13]_____ )  
        tmp *= [14]_____;  
    return tmp;  
}
```

Solution

```
double power1(double x, int n) {  
    // returns x * x * ... * x (n times)  
    int i;  
    double tmp;  
  
    for (i = 0, tmp = 1; i < n; i++)           // (12) tmp=1: 곱셈의 항등원으로 초기화  
                                                // (13) i++ : 반복할 때마다 카운터 증가  
        tmp *= x;                             // (14) x : 누적곱 연산 (tmp = tmp * x)  
  
    return tmp;  
}
```

1.5 2025 경북대 3-5번

5. 다음은 X좌표와 Y좌표로 구성되는 Point 형 구조체와 왼쪽 위, 오른쪽 아래의 Point 형 좌표로 구성되는 Rectangle 형 구조체를 이용해서 직사각형의 넓이를 구하는 프로그램이다.

```
/* 절대값 구하는 매크로 함수 */
#define abs(n) (((n) >= 0) ? ([15]_____) : ([16]_____))

typedef struct { double x, y; } Point;
struct Rectangle { Point p1, p2; };

void set_point(double x, double y) {
    Point temp;
    [17]_____ = x;  [18]_____ = y;
    return temp;
}

double area( struct Rectangle r ) {
    return abs( r.p1.x - [19]_____ )
           * abs( [20]_____ - r.p2.y );
}

int main(void) {
    struct Rectangle r;
    r.p1 = set_point(5.0, 6.0);
    r.p2 = set_point(10.0, 12.0);
    printf("넓이는 %.1f\n", area(r));  // 넓이는 24.0
    return 0;
}
```

Solution

```
/* 절댓값 매크로 함수 */
#define abs(n)    ((n) > 0 ? (n) : -(n))    // (15)(16): n이 양수면 그대로, 음
수면 부호 반전

typedef struct { double x, y; } Point;    // Point는 double형 x, y를 가진 사
용자 정의 자료형
struct Rectangle { Point p1, p2; };    // 두 Point로 구성된 Rectangle 구조체

Point set_point(double x, double y) {    // (주의) 반환형은 Point
    Point temp;
    temp.x = x;    // (17) temp.x: Point 구조체의 x 멤버
    temp.y = y;    // (18) temp.y: Point 구조체의 y 멤버
    return temp;
}

double area(struct Rectangle r) {
    return abs(r.p1.x - r.p2.x) *    // (19) r.p2.x: 가로 길이 = |x1 - x2|
           abs(r.p1.y - r.p2.y);    // (20) r.p1.y: 세로 길이 = |y1 - y2|
}

int main(void) {
    struct Rectangle r;
    r.p1 = set_point(5.0, 6.0);
    r.p2 = set_point(10.0, 12.0);
    printf("넓이는 %.1f\n", area(r));    // 출력: 넓이는 24.0
    return 0;
}
```

1.6 2024 경북대 3-1번

1. 다음은 정수 n 값의 자릿수(0인지, 1자리인지, 2자리 이상인지)를 판정해서 출력하는 프로그램의 일부분이다. 빈 칸을 채워 완성하여라. [문항당 1점]

```
_ (1) _____ ( n _ (2) _____ 0 )  
    printf("0입니다.\n");  
_ (3) _____ if ( n >= -9 _ (4) _____ n <= 9 )  
    printf("1자리입니다.\n");  
_ (5) _____  
    printf("2자리이상입니다.\n");
```

Solution

```
if ( n == 0 )          // (1)(2): n이 0과 같을 때 실행. ==는 비교 연산자.
    printf("0입니다.\n");

else if ( n >= -9 && n <= 9 )      // (3)(4): -9 이상 && 9 이하이면 한 자릿수.
    printf("1자리입니다.\n");

else                      // (5): 그 외의 모든 경우는 두 자릿수 이상.
    printf("2자리이상입니다.\n");

-
```

1.7 2024 경북대 3-2번

2. 다음은 자연수 w 값이 0, 1, 2면 “맑음”, “비”, “흐림”이라고 표시하는 프로그램의 일부
분이다. 빈 칸을 채워 완성하여라. [문항당 1점]

```
switch ( w ) {  
    (6)_____ : printf("맑음\n"); (7)_____ ;  
    (8)_____ : printf("비\n");    (7)_____ ;  
    (9)_____ : printf("흐림\n");  
}
```

Solution

```
switch ( w ) {  
    case 0:                                // (6) case 0: w가 0일 때 실행  
        printf("맑음\n");  
        break;                            // (7) break: 다음 case로 넘어가지 않고 switch 종료  
  
    case 1:                                // (8) case 1: w가 1일 때 실행  
        printf("비\n");  
        break;                            // (7) 동일하게 switch 종료  
  
    case 2:                                // (9) case 2: w가 2일 때 실행  
        printf("흐림\n");  
        break;  
}
```

-

1.8 2024 경북대 3-3번

3. 다음은 정수 $n1$ 부터 정수 $n2$ 까지의 값을 가로로(1숫자씩 빈칸 하나 띄워서) 출력하는 프로그램의 일부분이다. 빈 칸을 채워 완성하여라. [문항당 1점]

```
i = n1;
while ( (10)_____ ) {
    printf("__ (11) __", (12)___ );
    i++;
}
/* 출력예: n1 = 10, n2 = 13
   10 11 12 13
*/
```

Solution

```
    i = n1;

while ( i <= n2 ) {           // (10) i가 n2보다 작거나 같을 동안 반복
    printf("%d ", i);        // (11)(12) i의 값을 출력 (정수 서식 %d)
    i++;                     // i를 1 증가
}

/* 예시: n1 = 10, n2 = 13
   출력: 10 11 12 13
*/

-
```

1.9 2024 경북대 3-4번

4. 다음 프로그램 일부의 실행 결과를 나타내어라. [문항당 1점]

```
int i;  
for (i=2; i<5; i++)  
    printf("ABBA");
```

(13) (실행결과)

Solution

- 초기값: $i = 2$
- 조건식: $i < 5 \rightarrow i$ 가 5보다 작을 동안 반복
- 증가식: $i++ \rightarrow$ 한 번 반복할 때마다 1씩 증가

i 의 변화에 따른 실행:

반복 횟수	i 값	실행 내용	1	2	"ABBA" 출력
2	3	"ABBA" 출력	3	4	"ABBA" 출력

따라서 총 3번 반복되며, 한 번에 "ABBA"가 출력되므로 결과는 다음과 같다.

(13) 실행 결과: ABBAABBAABBA

1.10 2024 경북대 3-5번

5. 다음 프로그램 일부의 실행 결과를 적어라. [문항당 1점]

```
int x = 5, *ptr = &x;
printf("%d\n", 6 * (*ptr));    // (14)_____
*ptr += 3;
printf("%d\n", 7 * x);         // (15)_____
ptr = &x; *ptr = 8;
printf("%d\n", x);             // (16)_____
```

Solution

```
int x = 5, *ptr = &x;

printf("%d\n", 6 * (*ptr));    // (14) -----
*ptr += 3;
printf("%d\n", 7 * x);        // (15) -----
ptr = &x; *ptr = 8;
printf("%d\n", x);            // (16) -----
```

[해설]

- 초기값: $x = 5$, ptr은 x의 주소를 가리킴 $\rightarrow *ptr = 5$
- (14): $6 * (*ptr) = 6 * 5 = 30$
- (15): $*ptr += 3 \rightarrow x = 8$, 따라서 $7 * x = 56$
- (16): 다시 $*ptr = 8 \rightarrow x$ 는 여전히 8

[출력 결과]

30

56

8

1.11 2024 경북대 3-6번

6. 다음 프로그램의 구조체 xyz의 모든 원소를 비교하여 모두 같으면 0, 하나라도 다르면 1을 반환하는 함수 xyz_eq()를 완성하고 함수 printf의 출력을 적어라. [문항당 1점]

```
struct xyz { int x; long y; double z; };

int xyz_eq( struct xyz *a, struct xyz *b ) {
    if ( a->x (17)_____ b->x ) return 1;
    else if ( (18)_____ b->y ) return 1;
    else if ( (19)_____ b->z ) return 1;
    else return 0;
}

int main(void) {
    struct xyz a = {0,2,0.1}, b = {0,3,0.1}, c = {0,4,98.8};
    printf("(a==b)+(a==c) = %d\n",
           xyz_eq(&a, &b) + xyz_eq(&a, &c));    // (20)_____
}
```

Solution

```
struct xyz { int x; long y; double z; };

int xyz_eq(struct xyz *a, struct xyz *b) {
    if (a->x != b->x) return 1;      // (17) x 비교
    else if (a->y != b->y) return 1; // (18) y 비교
    else if (a->z != b->z) return 1; // (19) z 비교
    else return 0;
}

int main(void) {
    struct xyz a = {0,2,0.1}, b = {0,3,0.1}, c = {0,4,98.8};
    printf("(a==b)+(a==c) = %d\n",
           xyz_eq(&a, &b) + xyz_eq(&a, &c)); // (20)
}
```

[해설]

- xyz_eq는 포인터로 구조체를 받아 각 멤버를 비교.
- 하나라도 다르면 1을 반환, 모두 같으면 0을 반환.
- xyz_eq(&a, &b) → 1 (y 다름)
- xyz_eq(&a, &c) → 1 (y 다름)

[출력 결과]

(a==b)+(a==c) = 2

1.12 2023 경북대 3-1번

(1) int 형 변수 n1 = 15로 시작해서, 다음 각 구문들이 순차적으로 실행될 경우 printf 함수에 의해 출력되는 값을 적어라.

```
int n1 = 15;
```

```
printf("%d\n", n1++ );    // (1)_____
```

```
printf("%d\n", n1 -= 5 );// (2)_____
```

```
printf("%d\n", n1 *= 2 );// (3)_____
```

```
printf("%d\n", n1 /= 3 );// (4)_____
```

```
printf("%d\n", n1 %= 4 );// (5)_____
```

Solution

```
int n1 = 15;
```

```
printf("%d\n", n1++);    // (1) 후위 증가: 출력 후 1 증가 → 15 출력, n1 = 16
printf("%d\n", n1 -= 5); // (2) n1 = n1 - 5 → 16 - 5 = 11 출력, n1 = 11
printf("%d\n", n1 *= 2); // (3) n1 = n1 * 2 → 11 * 2 = 22 출력, n1 = 22
printf("%d\n", n1 /= 3); // (4) n1 = n1 / 3 → 22 / 3 = 7(정수 나눗셈) 출력, n1 = 7
printf("%d\n", ++n1);    // (5) 전위 증가: 1 증가 후 출력 → n1 = 8 출력
```

[출력 결과]

15

11

22

7

8

[핵심 요약]

- $n1++$ → 후위 증가: 사용 후 증가
- $++n1$ → 전위 증가: 먼저 증가 후 사용
- $--$, $*=$, $/=$ → 각각 $n1 = n1 - 5$, $n1 = n1 * 2$, $n1 = n1 / 3$ 와 동일
- 정수형 나눗셈은 소수점 이하를 버림 ($22 / 3 = 7$)

1.13 2023 경북대 3-2번

(2) 다음 프로그램 일부의 실행 결과를 적어라.

```
int i, j;
i = 1, j = 4;

while ( i >= 0 )
    printf("%d %d\n", i--, ++j);

for ( i = 2; i <= 4; i++ ) {
    for ( j = 0; j < i; j++ )
        printf("%d", j);
    printf("\n");
}
```

Solution

1) while문 동작

- 초기값: $i = 1, j = 4$
- 조건: $i \geq 0$ 일 때 반복
- 각 반복에서 $i--$ 는 후위 감소(출력 후 감소), $++j$ 는 전위 증가(증가 후 출력)

반복	출력식 ($i--$, $++j$)	출력결과	실행 후 값 (i, j)
1	1 5	1 5	$i=0, j=5$
2	0 6	0 6	$i=-1, j=6$

while문 출력:

1 5

0 6

2) for문 동작

- 외부 반복: $i = 2, 3, 4$
- 내부 반복: $j = 0$ 부터 $i-1$ 까지 증가하며 j 출력
- 각 i 마다 한 줄 출력 후 줄바꿈

i 값	출력결과
2	01
3	012
4	0123

for문 출력:

01

012

0123

1.14 2023 경북대 3-3번

(3) 다음 프로그램 일부의 실행 결과를 적어라.

```
int x = 5, Z[3] = {1,2,3}, *ptr = &x;

printf("%d\n", 4 * *ptr);           // (11)_____
printf("%d\n", 6 * (*ptr));        // (12)_____
*ptr += 3;
printf("%d\n", 7 * x);              // (13)_____
ptr = &x; *ptr = 8;
printf("%d\n", x);                  // (14)_____
ptr = Z+1; *ptr = 9;
printf("%d %d %d\n", Z[0], Z[1], Z[2]); // (15)_____
```

Solution:

- 초기 상태 $x = 5$, $Z = [1, 2, 3]$, $\text{ptr} \rightarrow x$

$$(11) 4 * * \text{ptr} = 4 * 5 = 20$$

(*ptr은 x의 값 5를 의미하므로 20 출력)

$$(12) 6 * (* \text{ptr}) = 6 * 5 = 30$$

(괄호 위치만 다를 뿐 동일한 의미 — 여전히 x는 5)

$$(13) * \text{ptr} += 3 \rightarrow x = 8, \text{ 따라서 } 7 * x = 56$$

(ptr이 x를 가리키므로 x의 값 자체가 바뀜)

$$(14) \text{ptr} = x; * \text{ptr} = 8; \rightarrow x \text{에 } 8 \text{ 대입}$$

(x는 여전히 8, 출력: 8)

$$(15) \text{ptr} = Z + 1; \rightarrow \text{ptr이 } Z[1] \text{을 가리킴 } * \text{ptr} = 9; \rightarrow Z[1] \text{ 값이 } 9 \text{로 변경됨}$$

메모리 상태:

$$Z[0] = 1$$

$$Z[1] = 9 \quad \leftarrow \text{수정됨}$$

$$Z[2] = 3$$

출력: 1 9 3

[최종 출력 결과]

20

30

56

8

1 9 3

1.15 2023 경북대 3-4번

(4) 다음 프로그램의 구조체 xyz의 모든 원소를 비교하여 모두 같으면 1, 하나라도 다르면 0을 반환하는 함수 xyz_eq()를 완성하고 함수 printf의 출력 2개를 적어라.

```
struct xyz { int x; long y; double z; };

int xyz_eq( struct xyz *a, struct xyz *b ) {
    if ( a->x (16)_____ b->x ) return 0;
    if ( a->y (17)_____ b->y ) return 0;
    if ( a->z (18)_____ b->z ) return 0;
    return 1;
}

int main(void) {
    struct xyz a = {0,1,0.1}, b = {0,1,0.1}, c = {0,1,98.8};

    printf("a == b = %d\n", xyz_eq(&a, &b)); // (19)_____
    printf("a == c = %d\n", xyz_eq(&a, &c)); // (20)_____
}
```

Solution

- 구조체 xyz는 세 개의 멤버를 가진다: x (int), y (long), z (double).
- 포인터 a, b는 구조체를 가리키므로, 멤버 접근 시 a->x, b->y처럼 화살표 연산자(->)를 사용한다.

메모리 구조 (개념적)

```
a : [ x=0 | y=1 | z=0.1 ]
b : [ x=0 | y=1 | z=0.1 ]
c : [ x=0 | y=1 | z=98.8 ]
```

함수 동작 과정

1. xyz_eq(a, b) 호출 - 각 멤버 비교: a->x == b->x, a->y == b->y, a->z == b->z → 모두 동일 - 조건문을 한 번도 통과하지 않으므로 마지막 return 1; 실행 - 출력: a == b = 1
2. xyz_eq(a, c) 호출 - x, y는 같지만 z는 다름 (0.1 ≠ 98.8) - 세 번째 if문에서 return 0; 실행 - 출력: a == c = 0

[최종 출력 결과]

```
a == b = 1
a == c = 0
```

[핵심 정리]

- -> 연산자는 “포인터가 가리키는 구조체의 멤버”에 접근할 때 사용한다. 예: a->x ≡ (*a).x
- 구조체 전체 비교는 불가능하므로, 각 멤버를 개별적으로 비교해야 한다.
- 하나라도 다르면 즉시 return 0;으로 종료한다.

1.16 2022 경북대 3-1번

(1) 다음의 프로그램의 출력을 쓰시오

```
int i, j, k;

for (i=1; i<sizeof(float); i++) {
    for (j=i; j<sizeof(double); j++) {
        for (k=0; k<i; k++) printf("#");
        for (; k<j; k++) printf("*");
        printf("\n");
    }
}
```

1.17 2022 경북대 3-2번

(2) 다음의 프로그램의 출력을 쓰시오

```
short A[4] = {103, 55, 8, 20}, *P[4], i;
```

```
for (i=0; i<4; i++) P[i] = A+3-i;
```

```
printf("%d ", (int)(P[3]-P[0]));
```

```
printf("%d ", *P[3]-*(P[0]));
```

```
printf("%d ", (int)((++P[2])-(P[1]--)));
```

```
printf("%d\n", (int)((*P[2])-(*P[1])));
```

Solution

- $\text{sizeof}(\text{float}) = 4$, $\text{sizeof}(\text{double}) = 8$ (바이트 단위 정수 상수)
- 따라서 i 는 1, 2, 3까지 반복된다.
- 각 반복에서 j 는 i 부터 7까지 증가하며, 각 줄마다 i 개의 #과 $(j-i)$ 개의 *를 출력한다.

루프 구조 분석

i	j 범위	# 개수	* 개수	출력 줄 수
1	1-7	1	0-6	7
2	2-7	2	0-5	6
3	3-7	3	0-4	5

[실제 출력 결과]

```
#
##
***
****
*****
*****
*****
##
***
****
*****
*****
###
****
*****
*****
#####
```

[핵심 정리]

- `sizeof()`는 컴파일 시점 상수이므로, 루프는 고정된 횟수만큼 수행된다.
- `k`는 각 줄마다 0부터 `j-1`까지 순차 증가하며 먼저 `#`를 `i`개, 그 뒤에 `*`를 `(j-i)`개 출력한다.
- 줄바꿈은 `printf("\n");`로 매 `j`마다 한 번 실행된다.

[참고] C 언어 주요 자료형의 `sizeof` 값 요약**1) 64비트 시스템 (현대 GCC / Clang 기준)**

자료형	<code>sizeof(...)</code> 결과	비고
<code>char</code>	1	1 byte = 8 bits
<code>unsigned char</code>	1	부호 없음
<code>short</code>	2	보통 16-bit 정수
<code>unsigned short</code>	2	
<code>int</code>	4	32-bit 정수
<code>unsigned int</code>	4	
<code>long</code>	8	LP64 시스템 (Linux, macOS) 기준
<code>unsigned long</code>	8	
<code>long long</code>	8	항상 64-bit 정수
<code>float</code>	4	IEEE 754 단정밀도 실수
<code>double</code>	8	IEEE 754 배정밀도 실수
<code>long double</code>	16	(컴파일러에 따라 12 16 bytes)
<code>void *</code>	8	64-bit 주소 공간
<code>char *, int *, ...</code>	8	모든 포인터 크기는 동일함
<code>size_t</code>	8	unsigned 정수형 (주소 공간 크기 기반)

2) 32비트 시스템 (옛 Windows / 임베디드 환경 기준)

자료형	sizeof(...) 결과	비고
char	1	동일
short	2	동일
int	4	동일
long	4	LP32 시스템에서는 long도 4 byte
float	4	동일
double	8	동일
long double	12 또는 16	컴파일러별 상이
void *	4	포인터 크기 = 주소 공간 크기
size_t	4	주소공간이 32-bit이므로

3) 요약 비교 (64비트 vs 32비트)

자료형	64-bit 시스템	32-bit 시스템
char	1	1
short	2	2
int	4	4
long	8	4
float	4	4
double	8	8
long double	16	12 또는 16
포인터()	8	4

[요약 정리]

- sizeof()는 컴파일 시점에 평가되는 “정수 상수”이며, 실행 중 계산되지 않는다.
- 64-bit 환경에서는 long과 포인터 크기가 8 byte로 확장된다.
- 구조체나 배열에도 적용 가능하며, 내부 패딩(padding)에 의해 예상보다 커질 수 있다.
- 예시: sizeof(int[5]) = 20, sizeof(char[10]) = 10.

1.18 2021 경북대 3-1번

(1) 다음의 프로그램의 출력을 쓰시오

```
char S[][30] = {"Crocodile", "cucumber"};
int i, j;
for(i=0; i<2; i++) {
    for(j=0; j<strlen(S[i]); j++) S[i][j]+=2;
    printf("%s\n", S[i]);
}
printf("\n");
```

Solution

이 프로그램은 2차원 문자 배열 S에 두 개의 문자열을 저장한 후, 각 문자의 ASCII 값을 모두 2씩 증가시켜 새로운 문자열로 만든다.

- 문자열 저장 구조:

char S[][30] 은 ‘문자열 2개, 각각 최대 30글자’의 2차원 배열을 의미한다. 첫 번째 차원(문자열 개수)은 초기값으로 자동 유추되지만, 두 번째 차원(한 문자열의 최대 길이)은 반드시 명시해야 한다.

- 내부 루프 동작:

S[i][j] += 2; 는 각 문자의 ASCII 코드에 2를 더하는 연산이다. 즉, ‘A’(65)는 ‘C’(67)가 되고, ‘c’(99)는 ‘e’(101)가 된다.

- 문자 변환 결과:

”Crocodile” → ”Etqeqfkng”

”cucumber” → ”ewewodgt”

- 최종 출력:

Etqeqfkng

ewewodgt

1.19 2021 경북대 3-2번

(2) 다음 수식을 계산하는 C언어 문장을 한 줄로 작성하라. 값 계산은 필요없다.

$$\sqrt[4]{e^{1.67} \cdot 1.65^{\log_2 8}}$$

(`#include <math.h>` 가 코드 첫줄에 이미 선언되어 있다고 가정한다)

[해설]

수식을 C 표준 수확함수로 치환하면 다음 한 줄로 표현된다.

```
double y = pow( exp(1.67) * pow(1.65, log2(8.0)), 0.25 );
```

설명:

- $e^x \rightarrow \exp(x)$, $a^b \rightarrow \text{pow}(a, b)$.
- $\log_2 x \rightarrow \log2(x)$ (C11의 `math.h`). (대체식: $\log(x)/\log(2.0)$)
- 4제곱근 $\sqrt[4]{X} \rightarrow \text{pow}(X, 0.25)$. (대체식: `sqrt(sqrt(X))`)
- 실수 연산을 명확히 하기 위해 정수 8은 8.0으로 표기.

1.20 2021 경북대 3-3번

(3) 다음의 프로그램의 출력을 쓰시오

```
short A[4] = {8, 4, 11, 9}, *P[4];  
for (i=0; i<4; i++) P[i] = A+i;  
  
printf("%d ", (int)(P[3]-P[2]));  
printf("%d ", *(P[1])-*(P[0]));  
printf("%d ", (int)((P[2]--)-(++P[1])));  
printf("%d\n", (int)((*P[2])-(*P[1])));
```

Solution

1. 배열과 포인터 초기화

i	A[i]	P[i] (주소)	*P[i] (값)
0	8	&A[0]	8
1	4	&A[1]	4
2	11	&A[2]	11
3	9	&A[3]	9

for문 이후에 $P[i] = A + i$; 이므로, $P[0] \rightarrow \&A[0]$, $P[1] \rightarrow \&A[1]$, $P[2] \rightarrow \&A[2]$, $P[3] \rightarrow \&A[3]$ 이다.

—

2. 각 printf문의 연산 과정

1. $(P[3] - P[2])$ 포인터 간의 차이는 ‘요소 간 거리’를 의미한다. $\&A[3] - \&A[2] = 1$ 이므로 결과는 1. (주의: 이는 값 9-11이 아니라, 주소 간의 거리이다.)

2. $*P[1] - *(P[0])$ $4 - 8 = -4$.

3. $(P[2]--) - (++P[1])$

- $++P[1]$ 수행 $\rightarrow P[1] = \&A[2]$
- $P[2]--$ 은 후위 감소 $\rightarrow$ 계산 시점에는 여전히 $\&A[2]$, 계산 후 $P[2] = \&A[1]$
- 계산 시점에서는 $\&A[2] - \&A[2] = 0$

결과: 0

4. $(*P[2]) - (*P[1])$ 위에서 $P[2] = \&A[1]$, $P[1] = \&A[2]$ 따라서 $A[1] - A[2] = 4 - 11 = -7$ 결과: -7

—

3. 최종 출력 결과

1 -4 0 -7

Tip:

- 포인터 - 포인터 는 ‘메모리 주소’ 차이가 아니라 ‘배열 인덱스 단위 거리’를 뜻한다.
- 반면 $*P - *Q$ 는 실제 ‘값의 차이’다.
- 전위/후위 연산자(++, --)가 섞인 포인터 연산은 “계산 시점”과 “갱신 시점”을 명확히 구분해야 한다.

1.21 2020 경북대 3-1번

(1) 다음의 프로그램의 출력을 쓰시오

```
char S[][30] = {"Crocodile", "cucumber"};
int i, j, k;
for(i=0; i<2; i++) {
    for(j=0, k=0; j<strlen(S[i]); j++)
        if ( S[i][j]=='c' ) k++;
    printf("%d", k);
}
printf("\n");
```

[해설]**1. 배열 구조**

`S[0] = "Crocodile", S[1] = "cucumber"`

2. 루프 동작

- 외부 루프: $i = 0, 1$ (두 문자열 순서대로)
- 내부 루프: 각 문자열의 문자 수만큼 순회하며, 문자가 'c'인 경우마다 $k++$ 수행.

3. 문자 비교 시 주의점

C 언어에서 문자 비교는 대소문자를 구분한다.

'C'(대문자)는 'c'와 다르므로 첫 문자열에서는 소문자 c 하나만 카운트된다.

4. 각 단계별 결과

- $i=0$: "Crocodile" → 소문자 c 1개 → $k=1$
- $i=1$: "cucumber" → 소문자 c 2개 → $k=2$

5. 최종 출력

12

1.22 2020 경북대 3-2번

(2) 다음의 프로그램의 출력을 쓰시오

```
short A[4] = {19, 21, 24, 27}, short *P[4], i;  
for (i=0; i<4; i++) P[i] = A+i;
```

```
printf("%d ", (int)(P[0]-P[3]));  
printf("%d ", *(P[0])-*(P[3]));  
printf("%d ", (int)((--P[2])-(P[1]++)));  
printf("%d\n", *P[2]-*P[1]);
```

[해설]**1. 초기 상태**

$P[0] \rightarrow \&A[0], P[1] \rightarrow \&A[1], P[2] \rightarrow \&A[2], P[3] \rightarrow \&A[3].$

2. 각 문장 계산

1. $(P[0] - P[3]):$ 포인터 간 거리(요소 수).

$$\&A[0] - \&A[3] = -3.$$

2. $*P[0] - *(P[3]):$ 값의 차이.

$$19 - 27 = -8.$$

3. $(--P[2]) - (P[1]++):$ 전위/후위 주의.

$$--P[2] \Rightarrow P[2] = \&A[1].$$

$P[1]++$ 는 연산 시점엔 $\&A[1]$ 을 사용하고, 이후 $\&A[2]$ 로 증가.

$$\text{계산 시점: } \&A[1] - \&A[1] = 0.$$

4. $*P[2] - *P[1]:$ 이제 $P[2] = \&A[1] \Rightarrow 21, P[1] = \&A[2] \Rightarrow 24. 21 - 24 = -3.$

3. 최종 출력

-3 -8 0 -3

1.23 2020 경북대 3-3번

(3) 다음의 프로그램의 출력을 쓰시오

```
int x, y, z;
x = 153; y = 85;

while(y!=0) {
    z = x % y;
    x = y;
    y = z;
}

printf("%d\n", x);
```

정답: 17.

[해설]

핵심: % 는 나머지 연산자이며, 위 루프는 유클리드 호제법으로 x 와 y 의 최대공약수(GCD)를 구한다. 루프 불변식은 “반복이 진행될수록 x 와 y 는 기존 쌍의 GCD를 보존한 채 더 작은 쌍으로 치환된다”이다.

반복 과정:

회차	x	y	$z = x \% y$
초기	153	85	—
1	153	85	$68 (\because 153 = 85 \cdot 1 + \mathbf{68})$
	$x \leftarrow 85$	$y \leftarrow 68$	
2	85	68	$17 (\because 85 = 68 \cdot 1 + \mathbf{17})$
	$x \leftarrow 68$	$y \leftarrow 17$	
3	68	17	$0 (\because 68 = 17 \cdot 4 + \mathbf{0})$
	$x \leftarrow 17$	$y \leftarrow 0$	

종료 조건: $y == 0$ 이 되면 루프 종료, 이때 x 가 GCD.

최종 출력:

17